MasterHub Raspberry Pi 5 Deployment Guide

Step by step installation and production operation

Prepared for the MasterHub team • 16 October 2026

Run MasterHub continuously on Raspberry Pi 5 and open its dashboard from a laptop or phone. This guide covers installation, the existing PostgreSQL connection, encrypted dashboard access, automatic startup, HTTPS access for the team, device connections and operational checks.

Start by keeping the existing PostgreSQL server and MQTT broker. Move the application first. An independent installation with PostgreSQL on the Pi requires a separate, verified database migration; the release ZIP does not contain your live database.

Deployment status

This is a procedure to execute on the Pi. It does not mean the Pi is already configured. Windows validation previously passed 250 backend tests and 17 frontend tests; ARM compatibility, HTTPS proxy changes and live hardware behavior still require validation on the actual Pi.

Example names used throughout

- Pi hostname: masterhub. Pi administrator: noushith. Replace the administrator name if you choose another account.
- Application directory: /opt/masterhub. Background service user: masterhub. These are separate from the browser operator login.
- Example Pi address: 192.168.1.50. Replace it with the address assigned by your router.
- Initial dashboard: http://127.0.0.1:15000/dashboard through an SSH tunnel.
- Team dashboard: https://masterhub.yourcompany.com/dashboard after DNS, certificates and the HTTPS proxy are configured. This is a placeholder hostname.

Before starting

Use a Raspberry Pi 5 with a reliable power supply, cooling, Ethernet where possible, and reliable storage. SSD storage is preferable for sustained database and log writes. Keep a current database backup outside the Pi. Plan a maintenance window before switching real devices from the Windows controller.

Follow steps 1 to 5 for the first working service, steps 6 and 7 for HTTPS and device cutover, and steps 8 and 9 for operations and troubleshooting.

1 Prepare and connect to the Pi

Use Raspberry Pi Imager to install Raspberry Pi OS Lite 64 bit. During imaging, set the hostname to masterhub, create your administrator account, select the timezone, and enable SSH. Prefer SSH key authentication. These steps do not require a desktop environment on the Pi.

- Connect Ethernet and power up the Pi. Reserve its IP address in the router so the address remains stable.
- The administrator account below is noushith. Use the account you actually created; there is no assumed default Pi password.

Run on Windows in PowerShell

```
ssh noushith@masterhub.local
```

If the hostname does not resolve, find the Pi in the router device list and connect using its IP address.

```
ssh noushith@192.168.1.50
```

Run on the Pi after connecting

```
hostname -I
uname -m
python3 --version
```

Confirm a 64 bit system with aarch64 architecture. For this procedure use Python 3.12 or newer. If the OS provides an older Python, resolve the OS or interpreter choice before continuing; do not overwrite the system Python installation.

Install the base packages

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip unzip curl
```

If the operating system requests a reboot, reboot and reconnect before continuing. Node.js is not required to serve the existing dashboard; it is only needed when running the frontend test suite.

Check before proceeding

- SSH access works from your laptop.
- You know the Pi hostname, reserved IP address and administrator account.
- The Pi has network access to obtain dependencies and reach the intended database and MQTT broker.

Reference: Raspberry Pi Getting started and remote access documentation, listed in step 10.

2 Transfer and install MasterHub

Use the compact release masterhub.zip from the project dist folder. Transfer source and static assets only. Windows virtual environments, Android SDKs, test caches and report previews do not belong in the Pi application installation.

Run on Windows in PowerShell

```
scp "D:\GALATICX\masterhub_iot_noushith\dist\masterhub.zip" `
  noushith@masterhub.local:~/masterhub.zip
```

The backtick at the end of the first line is PowerShell line continuation. Alternatively, enter the command on one line.

Run on the Pi for a new installation

```
sudo useradd --system --user-group \
  --home-dir /opt/masterhub \
  --shell /usr/sbin/nologin masterhub
sudo mkdir -p /opt/masterhub
sudo unzip ~/masterhub.zip -d /opt/masterhub
sudo chown -R masterhub:masterhub /opt/masterhub
```

If the service account or application directory already exists, inspect that installation instead of overwriting it. These commands assume a first installation. Keep the dedicated account without sudo privileges.

Create the Linux virtual environment

```
sudo -u masterhub python3 -m venv /opt/masterhub/.venv
sudo -u masterhub /opt/masterhub/.venv/bin/python \
  -m pip install --upgrade pip
sudo -u masterhub /opt/masterhub/.venv/bin/python \
  -m pip install -r /opt/masterhub/requirements-production.txt
sudo -u masterhub /opt/masterhub/.venv/bin/python \
  -m pip check
```

Expected dependency result: No broken requirements found. If a pinned dependency fails on ARM, stop and resolve that package error before enabling the service. Do not copy the Windows .venv to work around an installation failure.

Create the private configuration file

```
sudo -u masterhub cp /opt/masterhub/.env.example /opt/masterhub/.env
sudo chmod 600 /opt/masterhub/.env
sudo nano /opt/masterhub/.env
```

The service user owns this file because the existing Cortex settings feature can save credentials. Never commit it or include it in a shared release ZIP. Populate it using the next page.

3 Configure the database and operator login

Replace every placeholder below. Keep the application listening on loopback for SSH tunnel access and the later HTTPS proxy. The operator password must be a unique random value of at least 32 characters.

Contents to configure in the Pi .env file

```
MASTERHUB_HOST=127.0.0.1
MASTERHUB_PORT=5000
MASTERHUB_OPERATOR_USER=operator
MASTERHUB_OPERATOR_PASSWORD=REPLACE_WITH_LONG_RANDOM_PASSWORD

PGHOST=YOUR_EXISTING_DATABASE_SERVER_ADDRESS
PGPORT=YOUR_EXISTING_DATABASE_PORT
PGDATABASE=masterhub
PGUSER=masterhub_app
PGPASSWORD=YOUR_EXISTING_DATABASE_PASSWORD
DATABASE_URL=
MASTERHUB_INSTANCE_ID=YOUR_EXISTING_INSTANCE_ID

MQTT_HOST=YOUR_EXISTING_MQTT_BROKER
MQTT_PORT=1883
BCI_MOBILE_MQTT_HOST=YOUR_EXISTING_MQTT_BROKER
BCI_MOBILE_MQTT_PORT=1883
```

Generate a browser operator password on the Pi

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

Save the generated password securely and paste it into MASTERHUB_OPERATOR_PASSWORD. The example MQTT port assumes the current non-TLS broker arrangement; retain the actual broker settings and configure its security before production exposure.

Make the existing database reachable

localhost on the Pi means the Pi, not your Windows computer. Set PGHOST to the existing database server address reachable from the Pi and preserve the actual port. The database administrator may need to adjust the server listener, PostgreSQL access rules and firewall to allow the Pi specifically. Use the server's required TLS settings; never expose PostgreSQL publicly.

A nonempty DATABASE_URL overrides the PG fields. Preserve the current MASTERHUB_INSTANCE_ID so historical events remain associated with this installation. If PostgreSQL stays on Windows, that computer must remain running.

Validate before starting the app

```
cd /opt/masterhub
sudo -u masterhub .venv/bin/python tools/check_postgresql.py
```

Expected result: connectivity, table initialization and a write/read/rollback check pass. If the check fails, fix connectivity, credentials or permissions before continuing. Database migration to the Pi is a separate backup, restore and verification operation; do not substitute an empty database for existing data.

4 Start MasterHub and open the dashboard

Before starting against real devices, ensure the old MasterHub controller is stopped or use isolated test endpoints. Two controllers can conflict even when the dashboard looks normal.

Run on the Pi and leave the terminal open

```
cd /opt/masterhub
sudo -u masterhub .venv/bin/python serve.py
```

Use serve.py for the production server. Do not deploy using Flask debug mode or its development server. The process requires a working PostgreSQL connection and configured operator credentials.

Open an encrypted tunnel from Windows

Open a second Windows PowerShell window. Keep the first Pi session running. Enter:

```
ssh -N -L 15000:127.0.0.1:5000 noushith@masterhub.local
```

No output is normal while the tunnel is running. Leave this PowerShell window open. If local port 15000 is occupied, choose another free local port and use the same number in the browser address.

Open Chrome or Edge on your Windows laptop

http://127.0.0.1:15000/dashboard

Enter the operator username and password configured in the Pi .env file. The dashboard is served by the Pi; the laptop's loopback address forwards through the encrypted SSH connection. No Python installation is needed in the browser.

Check the service and data

- http://127.0.0.1:15000/healthz checks whether the web process responds.
- http://127.0.0.1:15000/api/health shows protected application health.
- Check Saved Activity in the dashboard to verify database access and persisted events.
- Confirm that connection indicators match the actual device state. An HTTP 200 response alone does not prove that hardware or storage is ready.

What happens when you close a window

Closing the tunnel disconnects the laptop browser. Closing the manual server session can stop the application. The next step replaces manual startup with a background service that survives SSH logout and starts at boot.

A monitor does not need to remain attached to the Pi. Phone access without a tunnel is covered by the HTTPS setup in steps 6 and 7.

5 Enable automatic startup with systemd

Stop the manually started server using Ctrl+C in its Pi terminal. Keep only one MasterHub process running; device connections and control state are held in memory.

Run on the Pi

```
sudo nano /etc/systemd/system/masterhub.service
```

Paste this service definition

```
[Unit]
Description=MasterHub control service
Wants=network-online.target
After=network-online.target
StartLimitIntervalSec=0

[Service]
Type=simple
User=masterhub
Group=masterhub
WorkingDirectory=/opt/masterhub
ExecStart=/opt/masterhub/.venv/bin/python /opt/masterhub/serve.py
Environment=PYTHONUNBUFFERED=1
Restart=on-failure
RestartSec=10
TimeoutStopSec=30
KillSignal=SIGTERM
NoNewPrivileges=true
UMask=0077

[Install]
WantedBy=multi-user.target
```

The app loads its own .env file. This service retries a failed startup every 10 seconds, which allows recovery when the database is temporarily unavailable. It does not prove readiness or replace monitoring.

```
sudo systemctl daemon-reload
sudo systemctl enable --now masterhub
sudo systemctl status masterhub --no-pager
```

Expected result: Active: active (running). The existing SSH tunnel can now open the same dashboard even after the server SSH session is closed.

Reboot check

```
sudo reboot
```

After reconnecting, run systemctl is-enabled masterhub, systemctl is-active masterhub, and curl http://127.0.0.1:5000/healthz. Reopen the laptop SSH tunnel after a Pi reboot.

6 Configure HTTPS access for the team

For browser access from laptops and phones without SSH tunnels, use Nginx with HTTPS. First obtain a real hostname, DNS pointing to the Pi's reserved address, and a certificate trusted by the team devices. Replace masterhub.yourcompany.com and the certificate paths below. Do not publish the example hostname.

Required application change before using the proxy

The current origin protection must see the original HTTPS scheme. Otherwise the dashboard can load while button actions fail with a cross-origin error. Add these arguments to the existing Waitress serve call in serve.py; this guide does not apply the change automatically:

```
trusted_proxy="127.0.0.1",
trusted_proxy_headers={"x-forwarded-proto"},
```

Keep MASTERHUB_HOST=127.0.0.1. Trust only the local proxy for this header and preserve operator authentication. Validate this change with login and a safe state-changing request.

Run on the Pi

```
sudo apt install -y nginx
sudo nano /etc/nginx/sites-available/masterhub
```

Paste after providing the certificate files

```
server {
    listen 443 ssl;
    server_name masterhub.yourcompany.com;
    ssl_certificate /etc/nginx/tls/masterhub-fullchain.pem;
    ssl_certificate_key /etc/nginx/tls/masterhub-key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    client_max_body_size 1m;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection "";
        proxy_buffering off;
        proxy_read_timeout 3600s;
    }
}
```

Keep the private key restricted to the administrator and Nginx service requirements. Disabling buffering supports the live event stream. The long read timeout allows quiet streaming connections; validate stream reconnects in your environment.

7 Enable team access and reconnect devices

Enable the HTTPS site on the Pi

```
sudo ln -s /etc/nginx/sites-available/masterhub \
  /etc/nginx/sites-enabled/masterhub
sudo nginx -t
sudo systemctl restart masterhub
sudo systemctl enable nginx
sudo systemctl reload nginx
```

If the site link already exists, do not recreate it. If nginx -t fails, fix the hostname, syntax or certificate paths before reloading. Allow HTTPS from the intended network and retain SSH access. Keep application port 5000 and PostgreSQL closed to ordinary browser clients.

Where the team opens the dashboard

https://masterhub.yourcompany.com/dashboard

Replace this placeholder with the real configured hostname. Open it in a laptop or phone browser on the permitted network and enter the operator credentials. For access outside the office, use the organisation's VPN unless a public deployment is deliberately designed and reviewed.

Reconnect each endpoint

- Windows PC: keep the PC agent running in its interactive Windows session. Select that PC as the target. Local Host now means the Pi and cannot run Windows Notepad or Windows desktop automation.
- Android: point the companion app at the intended broker and registration endpoint. Native control clients need operator HTTP Basic credentials; Android account endpoints retain their existing bearer-token contract.
- IoT hardware: retain exact MQTT topics and broker settings. Saved .runtime/hardware.json can override environment defaults. Review device-specific MQTT authentication and permissions before production exposure.
- USB or UART: reconnect the device and select its Linux serial port. Use a persistent device identity where possible rather than assuming COM port names carry over.

```
sudo usermod -aG dialout masterhub
sudo systemctl restart masterhub
```

Only add serial access if required. EMOTIV Cortex must be configured separately: localhost:6868 now refers to the Pi. Confirm headset, OS and Cortex compatibility, or use the supported remote Cortex connection. Do not simply disable TLS verification to connect to a remote host.

Cut over without losing runtime data

Stop actuators safely, then stop the old Windows controller before making the Pi the live controller. Preserve MASTERHUB_INSTANCE_ID, data/event-outbox and relevant .runtime/hardware.json settings. Transfer pending events only while the old writer is stopped. Keep backups; the release ZIP excludes live data.

8 Operate and verify the production service

Service commands on the Pi

```
sudo systemctl status masterhub --no-pager
sudo systemctl restart masterhub
sudo systemctl stop masterhub
sudo systemctl start masterhub
sudo journalctl -u masterhub -n 100 --no-pager
sudo journalctl -u masterhub -f
```

Press Ctrl+C to stop following logs; that does not stop the background service. Restart after changing .env. Run daemon-reload as well if the service definition changes.

Production acceptance sequence

1. Confirm dashboard login, CSS loading and accurate device status through the final access method.

2. Check Saved Activity and verify that a new event is persisted in PostgreSQL.

3. Issue a safe command to the intended device and confirm its acknowledgement and actual behavior.

4. Restart MasterHub and verify that stale actuator commands are not replayed.

5. Reboot the Pi and confirm automatic startup, dashboard access and device reconnection.

6. Test database and MQTT outages in a controlled window. Confirm visible failures, queued event handling and recovery.

7. Run a sustained workload representative of the expected devices and browser sessions. Watch CPU, memory, temperature, disk use and command latency.

8. Restore a database backup into an isolated test database and verify the recovered records. A backup file existing is not proof that restoration works.

Backups and updates

Schedule database backups on the database host and copy them to separate storage. Preserve private configuration securely, the event outbox, hardware settings and installation ID. Never package credentials into an ordinary release.

For each update: pass tests, keep the previous release, stop actuators, back up data, stop the service, install the new verified release, preserve configuration and pending events, restart, and perform the acceptance checks. Avoid overwriting a running installation.

If an update fails, stop the new process and restore the previous application release and compatible configuration. Database schema changes require their own rollback plan. Do not roll back by deleting queued events or replacing the active database with an older copy without reconciling new data.

9 Troubleshoot common problems

SSH cannot find the Pi

Use its actual IP address instead of masterhub.local. Confirm the Pi is powered on, SSH was enabled, and both machines can reach the same network. Guest Wi-Fi may isolate devices.

The dashboard does not open through the tunnel

Keep the tunnel PowerShell window open. Confirm the local port is free, the service is active, and the Pi can answer curl http://127.0.0.1:5000/healthz. The browser uses port 15000 on the laptop, while the Pi listener uses port 5000.

The service repeatedly restarts

```
sudo journalctl -u masterhub -n 100 --no-pager
```

Look for missing credentials, an operator password shorter than 32 characters, unreachable PostgreSQL, dependency import failures, or port 5000 already in use. Stop any manual serve.py process before starting the service.

Database connection fails

Check the real server address and port, database name, role permissions, password and TLS configuration. localhost cannot reach a database on another computer. Confirm that DATABASE_URL is not unexpectedly overriding the PG fields and that the server allows the Pi through its firewall and PostgreSQL access rules.

HTTPS loads but buttons fail

Confirm that the trusted-proxy arguments were added to Waitress and Nginx forwards the original Host and X-Forwarded-Proto. Validate the proxy configuration, restart MasterHub and test a safe mutation. Do not remove cross-origin protection to hide the error.

Certificate warning in the browser

Use the hostname covered by the certificate. Check the full certificate chain, certificate expiry, DNS and client trust. An internal certificate authority must be trusted on every intended client. Do not instruct operators to bypass certificate warnings.

Devices appear offline or desktop commands fail

Verify the broker address, topics, credentials and saved hardware settings. Ensure the PC or phone agent is running and selected. Windows automation needs the Windows agent and an interactive session; the Pi Local Host target does not provide it.

Live activity stops updating

Check the event stream connection, proxy buffering and timeouts, and browser reconnection behavior. The bounded Waitress thread pool is intended for a small operator group. Measure concurrent streams before expanding the deployment.

10 References and handover checklist

These sources support the platform setup and deployment concepts. Project-specific paths, credentials and device mappings must be checked against the actual installation.

Raspberry Pi setup and imaging

https://www.raspberrypi.com/documentation/computers/getting-started.html

Raspberry Pi hardware and power guidance

https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

Flask production deployment

https://flask.palletsprojects.com/en/stable/deploying/

Flask Waitress deployment

https://flask.palletsprojects.com/en/stable/deploying/waitress/

Nginx HTTPS configuration

https://nginx.org/en/docs/http/configuring_https_servers.html

Nginx proxy configuration

https://nginx.org/en/docs/http/ngx_http_proxy_module.html

PostgreSQL Debian packages and ARM64 support

https://www.postgresql.org/download/linux/debian/

EMOTIV Cortex platform requirements

https://emotiv.gitbook.io/system-requirements/cortex-api

systemd service reference

https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html

Record before handing the service to the team

- Pi hostname and reserved address; administrator and service owner.
- Final dashboard URL; certificate issuer and renewal owner.
- PostgreSQL host, port and backup owner; restore test date.
- MQTT broker and device topic owners; Cortex host and headset configuration.
- Release version, deployment date, acceptance results and rollback location.
- Credential storage location and recovery procedure, without writing passwords into this guide.

Keep the Pi service account unprivileged, restrict network access, and monitor storage and connection failures. A single Pi remains a single point of failure; define the expected recovery time and safe device behavior if it loses power.